

HRC2026 Training Program

Module 5: Perception for Industrial Manipulation

*Phiên bản: v1.0 FULL | Định dạng tự học + demo baseline + lab đo được
Đối tượng: sinh viên năm 3 CNTT/KHMT, đã hoàn thành Module 1-4*

Mục tiêu của module

Sau module này, sinh viên có thể thiết kế và triển khai perception pipeline cho thao tác công nghiệp: từ ảnh RGB-D hoặc video đến object class, mask/bounding box, centroid, orientation, pose, velocity/tracking và confidence để planner/motion module sử dụng được.

Thông tin	Nội dung
Thời lượng đề xuất	2 đến 2.5 ngày học tập trung; 10-14 giờ tự học/lab.
Guild chủ lực	Perception/Pose/Tracking; Manipulation và Evaluation cần học đủ để hiểu contract đầu vào/đầu ra.
Liên hệ HRC2026	Task 1: desktop sorting; Task 2: conveyor sorting; Task 3: workpiece insertion; Task 4: carton sealing.
Triết lý	Không chỉ tạo bounding box đẹp. Perception phải xuất dữ liệu hành động được: pose, velocity, confidence, grasp_hint/slot_hint và failure_reason.
Output cuối module	perception_interface.json, perception_report.csv, overlay images, mask/pose/tracking metrics, mini-project report.

Mục lục

1. Chuẩn đầu ra và bản đồ năng lực
2. Phần A - Lý thuyết tối thiểu
3. Phần B - Demo trên baseline HRC2026
4. Phần C - Bài lab có output đo được
5. Mini-project cuối module và rubric
6. Troubleshooting checklist
7. Phụ lục code skeleton và schema
8. Tài liệu tham khảo và video/tutorial mở

1. Chuẩn đầu ra và bản đồ năng lực

Mã	Sau module này sinh viên làm được gì?	Minh chứng đo được
M5-CLO1	Phân biệt classification, detection, segmentation, pose estimation, tracking và chọn đúng kỹ thuật cho từng task.	Bảng task-to-method trong report.
M5-CLO2	Xây pipeline xử lý ảnh cơ bản: load frame, preprocess, threshold/mask/contour, tính centroid/orientation.	overlay_centroid.png, classical_perception_report.csv.
M5-CLO3	Dùng hoặc chuẩn bị dữ liệu để train detector/segmenter như YOLO/SAM-style pipeline.	dataset_inventory.csv, label_quality_report.md.
M5-CLO4	Từ RGB-D tính được 3D centroid/pose thô trong camera/base frame, kế thừa kiến thức Module 4.	pose_estimator_report.csv.
M5-CLO5	Theo dõi vật chuyển động trên băng chuyền bằng tracker/predictor đơn giản.	tracker_report.csv, velocity_error_px_per_frame.
M5-CLO6	Thiết kế output contract cho planner/motion: object_id, class_id, pose, velocity, confidence, grasp_hint.	perception_interface.json.
M5-CLO7	Đánh giá perception bằng metric phù hợp: precision, recall, IoU, mAP, pose error, latency và end-to-end impact.	evaluation_summary.md.

Điểm nối với các module trước

Module 3 đã cung cấp frame, transform, SE(3), quaternion và FK/IK. Module 4 đã cung cấp camera intrinsic/extrinsic, RGB-D và camera-to-base transform. Module 5 dùng các kiến thức đó để tạo perception output có thể đưa trực tiếp vào planner/motion module.

2. Phần A - Lý thuyết tối thiểu

2.1. Perception trong industrial manipulation là gì?

Perception trong bài toán thao tác công nghiệp không dừng ở việc “nhận dạng ảnh”. Nó là tầng chuyển đổi quan sát thô từ camera/sensor thành trạng thái có cấu trúc để robot có thể ra quyết định và hành động. Với HRC2026, perception phải trả lời ít nhất 6 câu hỏi:

- ☐ Có những vật nào trong scene?
- ☐ Mỗi vật thuộc loại nào?
- ☐ Vật nằm ở đâu trong ảnh, trong 3D camera frame và trong robot/world frame?
- ☐ Vật đang đứng yên hay chuyển động? Nếu chuyển động, vận tốc và hướng là gì?
- ☐ Độ tin cậy của nhận dạng/pose/tracking là bao nhiêu?
- ☐ Planner/motion nên gấp ở đâu, theo hướng nào, hay nên bỏ qua vật đó?

Loại output perception	Ví dụ	Dùng cho task nào?
Class label	workpiece_A / workpiece_B / carton_lid / foam_slot	Tất cả task.
Bounding box 2D	[x1, y1, x2, y2]	Detection nhanh, tracking, debug overlay.
Mask/segmentation	Pixel thuộc về vật thể	T1/T3 rất hữu ích để tính centroid/orientation; T4 để tìm nắp carton.
Centroid 3D	[x, y, z] trong camera/base frame	Gấp/đặt/align.
Orientation/Pose	yaw hoặc SE(3)	Gấp đúng hướng, chèn vào foam, đóng nắp carton.
Velocity/track	vx, vy, confidence, object_id	Task 2 băng chuyền.

Loại output perception	Ví dụ	Dùng cho task nào?
Confidence/failure reason	low_mask_quality, occluded, out_of_workspace	Planner quyết định retry/fallback.

2.2. Bản đồ kỹ thuật: từ đơn giản đến nâng cao

Mức	Kỹ thuật	Ưu điểm	Hạn chế	Nên dùng khi
Level 0	Rule-based thresholding: màu, depth range, contour	Rất nhanh, dễ debug, không cần data lớn	Kém robust nếu lighting/texture thay đổi	Scene mô phỏng ổn định, màu/vật thể khác biệt rõ.
Level 1	Classical CV + RGB-D geometry: contour, PCA, minAreaRect, 3D centroid	Tạo pose/grasp hint tốt, ít training	Cần tuning theo scene	T1/T3 cần pose chính xác.
Level 2	Detector/segmenter học máy: YOLO detect/segment, Mask R-CNN, SAM-assisted labeling	Robust hơn với biến thiên pose/texture	Cần dữ liệu/label, latency cao hơn	T1/T2 hoặc khi rule-based không đủ.
Level 3	6D pose/template/ICP/FoundationPose	Ước lượng pose đầy đủ, phù hợp CAD/object pose	Thiết lập phức tạp, cần mesh/reference tốt	T3 insertion, gấp vật có hình học đặc thù.
Level 4	VLM/embodied perception hỗ trợ	Giúp mô tả scene, validate nhãn, phân tích lỗi	Có thể chậm/hallucinate/khó tái lập	Dùng offline/fallback, không nên phụ thuộc real-time.

2.3. Perception interface: hợp đồng giữa Perception và Planner

Để tích hợp tốt, perception module không nên trả về nhiều format tùy tiện. Cần chuẩn hóa một schema chung để planner/motion module dùng được. Một object observation nên có cấu trúc như sau:

```
{
  "frame_id": 128,
  "timestamp": 12.833,
  "camera_name": "front_rgbd",
  "objects": [
    {
      "object_id": "track_003",
      "class_id": "workpiece_A",
      "confidence": 0.92,
      "bbox_xyxy": [345, 210, 420, 292],
      "mask_quality": 0.86,
      "centroid_px": [382.5, 251.0],
      "centroid_camera_m": [0.12, -0.08, 0.74],
      "pose_base": {
        "position_m": [0.48, -0.16, 0.83],
        "quaternion_xyzw": [0.0, 0.0, 0.707, 0.707]
      },
      "velocity_px_per_frame": [2.3, 0.1],
      "grasp_hint": {
        "approach_axis": "z_down",
        "grasp_width_m": 0.045,
        "yaw_rad": 1.57
      },
      "failure_reason": null
    }
  ]
}
```

Nguyên tắc

Planner không nên phải biết object được detect bằng YOLO, threshold hay template matching. Planner chỉ cần nhận object state sạch, có confidence và failure_reason.

2.4. Image representation và preprocessing

Khái niệm	Ý nghĩa	Lưu ý thực hành
RGB/BGR	Ảnh màu 3 kênh. OpenCV mặc định đọc ảnh theo BGR.	Nếu overlay màu sai, kiểm tra RGB/BGR conversion.
HSV	Không gian màu tách hue/saturation/value, hữu ích cho threshold theo màu.	Hue trong OpenCV thường có range 0-179.
Depth	Mỗi pixel có khoảng cách đến camera.	Kiểm tra đơn vị: meter, millimeter hoặc normalized.
Binary mask	Ảnh 0/1 hoặc 0/255 biểu diễn vùng vật thể.	Morphology open/close giúp giảm nhiễu.
Contour	Biên của vùng mask.	Dùng để tính area, centroid, minAreaRect, orientation.
Point cloud	Tập điểm 3D suy ra từ depth + intrinsic.	Cần lọc outlier, NaN, far depth.

```
# Minimal preprocessing skeleton
import cv2 as cv
import numpy as np

rgb = cv.imread("frame.png")          # BGR by default
hsv = cv.cvtColor(rgb, cv.COLOR_BGR2HSV)
mask = cv.inRange(hsv, lowerb=(20, 60, 60), upperb=(40, 255, 255))
mask = cv.morphologyEx(mask, cv.MORPH_OPEN, np.ones((5,5), np.uint8))
contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
```

2.5. Classification, detection, segmentation, pose estimation và tracking

Bài toán	Input	Output	Metric chính	Ví dụ trong HRC2026
Classification	Crop/ảnh	Class label	Accuracy, precision/recall	Phân biệt workpiece A/B nếu đã crop vật.
Object detection	Ảnh RGB	BBox + class + confidence	mAP, precision/recall, FPS	Tìm vật trên bàn hoặc băng chuyền.
Instance segmentation	Ảnh RGB/RGB-D	Mask từng vật	IoU, mask mAP	Tính centroid/orientation chính xác hơn bbox.
Pose estimation	Ảnh + depth/CAD/keypoints	Vị trí + hướng/SE(3)	Position error, angular error, reprojection error	Gắp đúng yaw; chèn chi tiết vào foam.
Tracking	Video frames	Track ID + trajectory + velocity	ID switches, MOTA/HOTA, prediction error	Dự đoán điểm gắp trên băng chuyền.

2.6. Object detection và YOLO-style workflow

Detector phù hợp khi scene có nhiều vật và cần biết “vật nào ở đâu”. Với công cụ như YOLO, quy trình thực dụng là: chuẩn bị ảnh/label, chia train/val/test theo episode/scene, fine-tune từ pretrained weights, đánh giá bằng mAP/precision/recall/latency, rồi kiểm thử end-to-end trong simulation.

```
# Example YOLO-style dataset structure
perception_dataset/
  images/
```

```

train/
val/
test/
labels/
train/
val/
test/
data.yaml

# One label file per image in YOLO detection format:
# class_id x_center y_center width height (normalized 0-1)
0 0.518 0.431 0.083 0.120
1 0.244 0.612 0.077 0.106

```

Cảnh báo về chia dữ liệu

Không nên trộn ngẫu nhiên từng frame liên tiếp vào train/val/test. Với video/episode, các frame gần nhau rất giống nhau. Nên chia theo episode/seed/scene để tránh data leakage.

2.7. Segmentation: vì sao mask quan trọng hơn bbox trong thao tác?

Bounding box thường đủ cho nhận dạng thô, nhưng thao tác robot cần vị trí và orientation chính xác. Mask cho phép tính centroid theo vùng vật thật, loại bỏ background và tính orientation qua PCA/minAreaRect. Với vật dài/hẹp, bbox axis-aligned có thể gây sai yaw; mask hoặc rotated rectangle hữu ích hơn.

```

# Orientation from contour using minAreaRect
cnt = max(contours, key=cv.contourArea)
rect = cv.minAreaRect(cnt) # ((cx, cy), (w, h), angle_deg)
(cx, cy), (w, h), angle = rect
box = cv.boxPoints(rect).astype(int)

```

2.8. Pose estimation từ RGB-D: centroid, orientation và 6D pose

Trong Module 4, sinh viên đã học pixel + depth → point 3D. Trong Module 5, ta dùng công thức đó trên nhiều pixel thuộc mask để tạo point cloud của vật. Từ point cloud có thể tính centroid 3D, trục chính bằng PCA, hoặc align với template/CAD bằng ICP để refinement.

Mức pose	Cách làm	Output	Khi nào đủ?
Pose thô 2.5D	Centroid mask + depth median + yaw 2D từ minAreaRect	x,y,z + yaw	Gắp vật nằm phẳng trên bàn.
Pose 3D từ point cloud	Lấy point cloud trong mask, PCA trục chính, lọc outlier	centroid 3D + axes	Vật có hình dạng rõ, depth tốt.
PnP từ keypoints	Biết keypoint 3D trên vật và 2D projection	rvec/tvec object-to-camera	Vật có keypoint/corner rõ hoặc marker.
ICP/template alignment	Align point cloud đo được với CAD/template	Refined SE(3)	Chèn chính xác, foam slot, CAD có sẵn.
FoundationPose/6D model	Dùng CAD/reference image và model 6D pose	6D pose + tracking	Khi cần pose tốt nhưng classical khó.

```

# Compute 3D centroid from mask + depth + intrinsics
ys, xs = np.where(mask > 0)
z = depth[ys, xs]
valid = np.isfinite(z) & (z > 0.1) & (z < 2.0)
xs, ys, z = xs[valid], ys[valid], z[valid]
X = (xs - cx) * z / fx
Y = (ys - cy) * z / fy
Z = z
centroid_cam = np.array([np.median(X), np.median(Y), np.median(Z)])

```

2.9. Tracking và prediction cho băng chuyền

Task 2 có băng chuyền tốc độ cố định. Vì vậy perception không chỉ detect vị trí hiện tại mà còn phải dự đoán vị trí tương lai để planner pre-position robot. Cách đơn giản và hiệu quả là giữ track ID, ước lượng vận tốc trung bình theo pixel hoặc world coordinate, rồi ngoại suy đến thời điểm robot có thể gấp.

Thành phần	Cách làm đơn giản	Metric
Detection per frame	YOLO/threshold/segmentation để lấy bbox hoặc centroid	Precision/recall, latency.
Association	Ghép detection mới với track cũ bằng nearest centroid/IoU	ID switch count.
Velocity estimation	$v = (p_t - p_{t-k}) / k$ hoặc linear regression	Velocity error.
Prediction	$p_{future} = p_{now} + v * \Delta t$	Prediction error tại grasp line.
Decision gate	Nếu confidence thấp, vật ngoài vùng reachable, hoặc quá muộn thì skip	False grasp attempts, drop count.

```
# Constant-velocity predictor in image space
class SimpleTrack:
    def __init__(self, track_id):
        self.track_id = track_id
        self.history = [] # list of (frame_id, x, y)

    def update(self, frame_id, x, y):
        self.history.append((frame_id, x, y))
        self.history = self.history[-10:]

    def predict(self, target_frame):
        if len(self.history) < 2:
            return None
        f0, x0, y0 = self.history[0]
        f1, x1, y1 = self.history[-1]
        dt = max(f1 - f0, 1)
        vx, vy = (x1 - x0) / dt, (y1 - y0) / dt
        return x1 + vx * (target_frame - f1), y1 + vy * (target_frame - f1)
```

2.10. Confidence, failure detection và fallback

Trong robotics, perception không nên “tự tin giả”. Nếu output sai nhưng confidence cao, planner/motion sẽ hành động sai. Do đó cần thiết kế confidence và failure_reason rõ ràng.

Tín hiệu rủi ro	Có thể phát hiện bằng	Fallback đề xuất
Low detection confidence	score < threshold	Chụp lại frame hoặc dùng detector khác.
Mask quá nhỏ/lớn	area ngoài range	Loại bỏ object hoặc gán failure_reason.
Depth không hợp lệ	NaN/zero/out-of-range	Dùng median vùng lân cận hoặc bỏ vật.
Pose không ổn định	yaw/centroid thay đổi mạnh qua frame	Temporal smoothing; yêu cầu nhiều frame nhất quán.
Occlusion/overlap	IoU cao giữa objects, mask chồng	Chọn vật ít bị che trước.
Out of workspace	pose_base nằm ngoài vùng robot với tới	Planner skip hoặc chờ vật vào vùng gấp.

2.11. Synthetic data và domain randomization trong Isaac Sim

Isaac Sim có thể tạo dữ liệu tổng hợp có nhãn như RGB, 2D/3D bounding boxes và segmentation masks thông qua Replicator/Object data generation. Với HRC2026, đây là lợi thế lớn vì assets, camera và scene đều có trong

mô phỏng. Tuy nhiên, dữ liệu tổng hợp vẫn cần validation bằng end-to-end success trong task, không chỉ metric detection.

Yếu tố randomize	Ví dụ	Tác dụng
Vị trí/pose vật thể	random x/y/yaw trong vùng bàn hoặc băng chuyền	Tăng robust với placement.
Lighting/material	brightness, color, texture nhẹ	Giảm overfit ánh sáng.
Camera pose/noise	small jitter, depth noise	Tăng robust perception-to-pose.
Distractors/occlusion	vật nền hoặc overlap nhẹ	Giúp detector xử lý hard cases.
Class balance	số lượng A/B cân bằng	Giảm bias classifier/detector.

2.12. Metric đánh giá perception

Metric	Dùng cho	Cách hiểu thực dụng
Precision	Detection/classification	Trong các vật model báo có, bao nhiêu là đúng? Precision thấp → gặp nhầm vật/background.
Recall	Detection/classification	Trong các vật thật, model tìm được bao nhiêu? Recall thấp → bỏ sót vật.
IoU	BBox/mask	Mức overlap giữa prediction và ground truth. Mask IoU giúp đo chất lượng vùng vật.
mAP	Detector/segmenter	Metric tổng hợp theo confidence/IoU thresholds; tốt cho model selection nhưng chưa đủ.
Centroid error	Pose thô	Sai số pixel hoặc meter giữa centroid dự đoán và ground truth.
Angular error	Orientation/pose	Sai số yaw/quaternion; quan trọng khi chèn/align.
Latency/FPS	Real-time perception	Nếu perception quá chậm, Task 2 sẽ miss vật.
End-to-end success	Toàn hệ thống	Metric cuối cùng: robot có hoàn thành task không?

2.13. Thiết kế perception theo 4 task HRC2026

Task	Perception cần xuất	Kỹ thuật khuyến nghị	Failure mode cần log
Task 1 Desktop Sorting	class_id, mask/bbox, centroid_base, yaw/grasp_hint	YOLO/segmentation + RGB-D centroid + minAreaRect/PCA	wrong_class, missed_object, bad_yaw, low_depth_quality
Task 2 Conveyor Sorting	class_id, track_id, centroid, velocity, predicted_intercept	Detector/tracker + constant velocity/Kalman + decision gate	late_detection, id_switch, prediction_error, out_of_reach
Task 3 Workpiece Insertion	part class/pose, slot pose, alignment error	Segmentation + depth + template/ICP/PnP; slot keypoints	wrong_slot, pose_offset, angle_error, occluded_slot
Task 4 Carton Sealing	carton pose, lid edge/keypoints, short/long side state	Keypoint/edge segmentation + state machine verification	lid_not_detected, wrong_side, uncertain_state, occlusion

3. Phần B - Demo trên baseline HRC2026

Mục tiêu demo

Sinh viên không chỉ đọc lý thuyết mà phải nhìn thấy perception gắn với baseline: camera stream, frame capture, object/asset paths, config, output overlay và JSON interface. Tất cả demo đều có thể chạy ở mức tối thiểu bằng frame mẫu nếu máy chưa chạy Isaac Sim.

3.1. Demo 0 - Tìm perception-related files trong baseline

Mục tiêu: lập bản đồ repo để biết camera, task config, dataset, assets và scripts nằm ở đâu. Sinh viên chạy các lệnh sau trong thư mục baseline hoặc bản clone của nhóm:

```
# Repository map for perception
find . -maxdepth 4 -type f | grep -Ei
"camera|rgb|depth|sensor|perception|detect|segment|track|config|yaml" | sort > repo_perception_files.txt
find . -maxdepth 4 -type f | grep -Ei "Part|Conveyor|Foam|Packing|Sorting|Box|yaml" | sort >
task_config_files.txt
find . -maxdepth 5 -type f | grep -Ei "usd|urdf|stl|obj|png|jpg|mp4|parquet" | sort >
asset_data_files.txt
```

Output	Tiêu chí pass
repo_perception_files.txt	Có ít nhất 10 file hoặc ghi rõ nếu baseline hiện không có module perception riêng.
task_config_files.txt	Tìm được config hoặc scene files liên quan đến 4 task.
asset_data_files.txt	Tìm được assets/dataset/media để dùng cho demo perception.
baseline_perception_map.md	Viết 1 trang mô tả camera/input/output liên quan perception.

3.2. Demo 1 - Camera stream inventory

Mục tiêu: xác định baseline có những camera nào, resolution, frame rate, loại dữ liệu RGB/depth/segmentation và cách lấy frame. Nếu chưa chạy Isaac Sim, sinh viên dùng dataset video/frame chính thức hoặc frame mẫu do nhóm trích xuất.

Trường thông tin	Ví dụ cần ghi
camera_name	front_rgb, wrist_left_rgb, wrist_right_rgb, overhead_cam...
modalities	rgb, depth, instance_segmentation, semantic_segmentation
resolution	640x480, 1280x720...
frame_id/timestamp	Có timestamp hoặc index frame không?
coordinate frame	camera frame tương ứng trong scene/baseline
access path	script/function/config để lấy frame

```
# Suggested output CSV columns
camera_name, modality, width, height, fps_or_dt, frame_name, source_file, notes
front_rgbd, rgb+depth, 640, 480, 30, front_camera, task_config.yaml, verify in baseline
```

3.3. Demo 2 - Classical perception baseline từ frame RGB-D

Mục tiêu: tạo một perception baseline không cần train: threshold/mask, contour, centroid, orientation, depth median. Đây là baseline cực kỳ quan trọng để debug trước khi dùng deep learning.

```
# demo_classical_perception.py (skeleton)
import cv2 as cv
import numpy as np
import json

rgb = cv.imread("sample_rgb.png")
```



```

depth = np.load("sample_depth.npy") # meters, shape HxW
hsv = cv.cvtColor(rgb, cv.COLOR_BGR2HSV)

# Example threshold; each team must tune from actual frames
mask = cv.inRange(hsv, np.array([10, 40, 40]), np.array([35, 255, 255]))
mask = cv.morphologyEx(mask, cv.MORPH_OPEN, np.ones((5,5), np.uint8))
contours, _ = cv.findContours(mask, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
objects = []
for i, cnt in enumerate(contours):
    area = cv.contourArea(cnt)
    if area < 100:
        continue
    M = cv.moments(cnt)
    if M["m00"] == 0:
        continue
    cx_px = M["m10"] / M["m00"]
    cy_px = M["m01"] / M["m00"]
    rect = cv.minAreaRect(cnt)
    objects.append({
        "object_id": f"obj_{i}",
        "class_id": "unknown_or_rule_based",
        "confidence": 0.5,
        "centroid_px": [cx_px, cy_px],
        "area_px": area,
        "yaw_image_deg": rect[2]
    })

with open("perception_classical_output.json", "w") as f:
    json.dump({"objects": objects}, f, indent=2)

```

3.4. Demo 3 - YOLO/SAM-assisted perception starter

Mục tiêu: sinh viên hiểu cách thay classical perception bằng detector/segmenter học máy. Không bắt buộc phải train model lớn trong module này; cần nắm format dữ liệu, lệnh inference, output và cách chuyển output sang perception_interface.json.

```

# Optional YOLO quickstart concept - adjust model path and dataset
pip install ultralytics

yolo predict model=yolo11n.pt source=sample_frames/ save=True
# For custom training after labels are ready:
yolo detect train model=yolo11n.pt data=hrc2026_parts.yaml imgsz=640 epochs=50 batch=16
# For segmentation:
yolo segment train model=yolo11n-seg.pt data=hrc2026_parts_seg.yaml imgsz=640 epochs=50

```

Licensing reminder

Ultralytics YOLO open-source package uses AGPL-3.0 for open-source use; teams should check competition and institutional constraints before final submission if shipping modified code/model.

3.5. Demo 4 - Conveyor tracker và intercept prediction

Mục tiêu: dùng detection centroids qua nhiều frame để tính velocity và predicted intercept. Đây là phần then chốt cho Task 2.

```

# Expected tracker output CSV
frame_id, track_id, class_id, cx_px, cy_px, vx_px_frame, vy_px_frame, pred_x_at_grasp_line,
pred_y_at_grasp_line, confidence
1, track_01, A, 310.4, 220.1, 2.1, 0.0, 500.0, 220.1, 0.91
2, track_01, A, 312.6, 220.0, 2.2, -0.1, 500.0, 219.8, 0.92

```

3.6. Demo 5 - Pose exporter cho planner/motion

Mục tiêu: gom output từ detector/segmenter/RGB-D transform thành một file JSON mà planner/motion có thể đọc. Nếu chưa có planner, sinh viên viết một script giả lập planner kiểm tra các điều kiện: confidence, reachable, selected object, failure_reason.

```
# demo_select_next_object.py
import json
obs = json.load(open("perception_output.json"))
valid = []
for obj in obs["objects"]:
    if obj["confidence"] < 0.6:
        continue
    if obj.get("failure_reason"):
        continue
    # Example reachable gate: x/y/z already in base frame
    pos = obj["pose_base"]["position_m"]
    if 0.2 <= pos[0] <= 0.8 and -0.5 <= pos[1] <= 0.5 and 0.5 <= pos[2] <= 1.2:
        valid.append(obj)
print("selected", valid[0]["object_id"] if valid else "none")
```

4. Phần C - Bài lab có output đo được

Quy tắc nộp lab

Mỗi lab phải có file output cụ thể. Không chấp nhận mô tả “đã chạy được” nếu không có CSV/JSON/ảnh overlay/log. Output phải đặt trong thư mục lab_outputs/module5/<student_name_or_team>/.

Lab 1 - Dataset/scene inventory cho perception

Mục tiêu: Tìm và mô tả dữ liệu/frames/assets có thể dùng để phát triển perception cho 4 task.

Output bắt buộc	Mô tả
dataset_inventory.csv	Nộp trong thư mục lab_outputs/module5/.
class_map.yaml	Nộp trong thư mục lab_outputs/module5/.
sample_gallery_contact_sheet.png	Nộp trong thư mục lab_outputs/module5/.
data_quality_notes.md	Nộp trong thư mục lab_outputs/module5/.

Tiêu chí pass:

- ☐ Liệt kê tối thiểu 20 ảnh/frame hoặc 5 video/episodes nếu có.
- ☐ Có class map cho workpiece A/B, foam slot, carton lid/edge.
- ☐ Có ghi rõ nguồn dữ liệu: baseline, assets, tự capture, synthetic.

Lab 2 - Classical mask, centroid và orientation

Mục tiêu: Từ 20 frame mẫu, tạo mask bằng threshold/contour, tính centroid và yaw_image.

Output bắt buộc	Mô tả
classical_perception_report.csv	Nộp trong thư mục lab_outputs/module5/.
overlay_centroid_*.png	Nộp trong thư mục lab_outputs/module5/.
mask_examples/*.png	Nộp trong thư mục lab_outputs/module5/.

Tiêu chí pass:

- ☐ Có ít nhất 20 dòng kết quả.
- ☐ Overlay phải thấy bbox/mask/centroid/orientation.
- ☐ Ghi latency trung bình/frame.

Lab 3 - Detection/segmentation starter

Mục tiêu: Chuẩn bị format dữ liệu YOLO-style hoặc chạy inference pretrained/fine-tune nhỏ.

Output bắt buộc	Mô tả
data.yaml	Nộp trong thư mục lab_outputs/module5/.
labels_sample/	Nộp trong thư mục lab_outputs/module5/.
detector_eval_summary.md	Nộp trong thư mục lab_outputs/module5/.
detection_overlay_*.png	Nộp trong thư mục lab_outputs/module5/.

Tiêu chí pass:

- ☐ Có train/val/test split hợp lý.
- ☐ Có confusion matrix hoặc precision/recall nếu có labels.
- ☐ Không trộn random frame liên tiếp nếu dữ liệu là video.

Lab 4 - RGB-D centroid và pose thô trong base frame

Mục tiêu: Dùng mask + depth + intrinsics/extrinsics để tính centroid_camera và centroid_base.

Output bắt buộc	Mô tả
pose_estimator_report.csv	Nộp trong thư mục lab_outputs/module5/.
transform_chain_report.json	Nộp trong thư mục lab_outputs/module5/.
overlay_3d_projection.png	Nộp trong thư mục lab_outputs/module5/.

Tiêu chí pass:

- ☐ Sai số reprojection hoặc sanity check phải được ghi.
- ☐ Có kiểm tra depth unit và invalid depth.
- ☐ Output schema tương thích perception_interface.json.

Lab 5 - Conveyor tracking/prediction

Mục tiêu: Từ video hoặc frame sequence, tạo track_id, velocity và prediction tại grasp line.

Output bắt buộc	Mô tả
tracker_report.csv	Nộp trong thư mục lab_outputs/module5/.
tracking_overlay.mp4 hoặc tracking_overlay_frames/	Nộp trong thư mục lab_outputs/module5/.
prediction_error_summary.md	Nộp trong thư mục lab_outputs/module5/.

Tiêu chí pass:

- ☐ Có ít nhất 3 tracks hoặc ghi rõ nếu data không đủ.
- ☐ Có velocity và predicted point.
- ☐ Có rule skip nếu object quá muộn/confidence thấp.

Lab 6 - Task-specific perception contract

Mục tiêu: Thiết kế output contract cho một trong 4 task và viết validator kiểm tra schema.

Output bắt buộc	Mô tả
perception_interface.json	Nộp trong thư mục lab_outputs/module5/.
validate_perception_schema.py	Nộp trong thư mục lab_outputs/module5/.
schema_validation_report.txt	Nộp trong thư mục lab_outputs/module5/.

Tiêu chí pass:

- ☐ Validator phát hiện thiếu field bắt buộc.
- ☐ Có failure_reason và confidence.
- ☐ Có ví dụ object hợp lệ và object bị reject.

5. Mini-project cuối module: Perception-to-Action Contract v0

Sinh viên hoặc nhóm 2-3 người chọn một task HRC2026 và xây một perception pipeline tối thiểu có thể chuyển từ ảnh/video/RGB-D sang output có cấu trúc cho planner/motion. Ưu tiên Task 1 hoặc Task 2 vì có thể đo nhanh bằng frame/video; nhóm khá có thể chọn Task 3/4.

Task lựa chọn	Yêu cầu tối thiểu	Output chính
Task 1	Detect/segment 4 workpieces, phân loại A/B, xuất centroid/yaw/grasp_hint.	perception_output_task1.json + overlay.
Task 2	Detect/track vật trên băng chuyền, xuất velocity/predicted_intercept.	tracker_report.csv + prediction overlay.
Task 3	Detect part + foam slot, xuất alignment/slot_hint.	slot_pose_report.csv + overlay.
Task 4	Detect carton/lid edge/keypoints, xuất state short/long lid.	carton_state_report.csv + overlay.

5.1. Cấu trúc thư mục nộp

```
module5_submission_<team_or_student>/
  README.md
  src/
    run_perception.py
    perception_schema.py
    tracking.py                # nếu chọn Task 2
    pose_from_rgbd.py         # nếu dùng RGB-D
  configs/
    class_map.yaml
    camera_config.yaml
    thresholds.yaml
  data_samples/
    images_or_frames/
    depth_or_video/           # nếu có
  outputs/
    perception_output.json
    perception_report.csv
    overlay_images/
    tracker_report.csv        # nếu có tracking
    pose_estimator_report.csv # nếu có RGB-D pose
    latency_report.csv
  report/
    module5_perception_report.md
```

5.2. Rubric chấm mini-project 100 điểm

Hạng mục	Điểm	Mô tả
Hiểu bài toán và thiết kế pipeline	15	Chọn đúng kỹ thuật cho task; có sơ đồ pipeline; nêu rõ input/output/failure modes.
Dữ liệu và annotation/label quality	15	Có dataset inventory, class map, split hợp lý, ghi rõ nguồn dữ liệu và chất lượng nhãn.
Implementation chạy được	20	Script chạy được bằng một lệnh; có config; không hard-code quá mức; output đúng schema.
Perception quality metrics	20	Có precision/recall/loU hoặc centroid/pose/tracking error; có latency; có phân tích lỗi.
Integration readiness	15	Output có thể đưa vào planner/motion: pose_base, confidence, grasp_hint/slot_hint, failure_reason.
Debug overlay và reproducibility	10	Có ảnh/video overlay, log, README rõ cách chạy.

Hạng mục	Điểm	Mô tả
Teamwork/documentation	5	Report ngắn gọn, phân công rõ, commit/log có ý nghĩa.

5.3. Ngưỡng đánh giá

Mức	Điểm	Ý nghĩa
Excellent	85-100	Có thể đưa vào sprint tích hợp Team A/B ngay.
Good	70-84	Pipeline tốt, cần cải thiện robustness hoặc metrics.
Pass	55-69	Có output chạy được nhưng chưa đủ tin cậy để tích hợp.
Needs revision	<55	Thiếu schema/metric/output hoặc không tái lập được.

6. Troubleshooting checklist

Triệu chứng	Nguyên nhân thường gặp	Cách kiểm tra/sửa
BBox/mask lệch khỏi vật	RGB-depth resize/crop không đồng bộ; sai color channel; sai coordinate convention.	Overlay từng bước: raw image, mask, bbox, centroid; kiểm tra image size và RGB/BGR.
Centroid 3D nhảy mạnh	Depth nhiễu/NaN; mask có background; vật bị occlusion.	Dùng median depth, lọc valid depth, erosion mask, temporal smoothing.
Yaw sai 90 độ	minAreaRect convention; w/h đảo; trục chính không xác định duy nhất.	Chuẩn hóa yaw theo cạnh dài; kiểm tra bằng overlay trục orientation.
Detector accuracy cao nhưng robot grasp sai	Metric perception không đo pose/grasp; frame transform sai; confidence không phản ánh chất lượng pose.	Thêm centroid/pose error, transform sanity, end-to-end lab.
Task 2 miss vật	Latency cao; prediction không tính thời gian robot di chuyển; track ID switch.	Đo FPS/latency, pre-position, skip late objects, association tốt hơn.
False positive background	Threshold/detector nhầm texture; class imbalance.	Thêm negative samples, area/depth gates, confidence threshold.
Model overfit synthetic data	Domain randomization chưa đủ; split bị leakage.	Randomize lighting/pose/camera; validate trên seed/scene khác.

7. Phụ lục code skeleton và schema

7.1. perception_schema.py

```
from dataclasses import dataclass, asdict
from typing import Optional, List, Dict, Any
import json

@dataclass
class PoseBase:
    position_m: List[float]
    quaternion_xyzw: List[float]

@dataclass
class PerceivedObject:
    object_id: str
    class_id: str
    confidence: float
    bbox_xyxy: List[float]
    centroid_px: List[float]
    centroid_camera_m: Optional[List[float]]
    pose_base: Optional[PoseBase]
    velocity_px_per_frame: Optional[List[float]]
    grasp_hint: Optional[Dict[str, Any]]
    failure_reason: Optional[str] = None

@dataclass
class PerceptionOutput:
    frame_id: int
    timestamp: float
    camera_name: str
    objects: List[PerceivedObject]

    def save(self, path: str):
```

```

def default(o):
    if hasattr(o, "__dataclass_fields__"):
        return asdict(o)
    raise TypeError(type(o))
with open(path, "w") as f:
    json.dump(self, f, default=default, indent=2)

```

7.2. validate_perception_output.py

```

import json, sys
required = ["object_id", "class_id", "confidence", "bbox_xyxy", "centroid_px"]
path = sys.argv[1]
obs = json.load(open(path))
errors = []
if "objects" not in obs:
    errors.append("missing objects")
for i, obj in enumerate(obs.get("objects", [])):
    for k in required:
        if k not in obj:
            errors.append(f"object {i}: missing {k}")
    if not (0.0 <= obj.get("confidence", -1) <= 1.0):
        errors.append(f"object {i}: confidence out of range")
    if len(obj.get("bbox_xyxy", [])) != 4:
        errors.append(f"object {i}: bbox must have 4 numbers")
print("PASS" if not errors else "FAIL")
for e in errors:
    print("-", e)

```

7.3. latency_report.py

```

import time
import pandas as pd

records = []
for frame_id in range(100):
    t0 = time.perf_counter()
    # run_perception(frame)
    # placeholder for student pipeline
    t1 = time.perf_counter()
    records.append({"frame_id": frame_id, "latency_ms": (t1 - t0) * 1000})

pd.DataFrame(records).to_csv("latency_report.csv", index=False)
print(pd.DataFrame(records)["latency_ms"].describe())

```


8. Tài liệu tham khảo và video/tutorial mở

Ưu tiên tài liệu chính thức, open documentation hoặc GitHub liên quan trực tiếp đến HRC2026, perception, detection, segmentation, pose estimation, tracking và synthetic data.

8.1. Tài liệu chính thức HRC2026/UBTECH

Tài liệu	Dùng để học gì?	Link
UBTECH HRC2026 Baseline GitHub	Baseline chính thức: simulation, LeRobot, teleoperation, record/replay, training/evaluation.	https://github.com/UBTECH-Robot/GlobalHumanoidRobotChallenge_2026_Baseline
UBTECH challenge2026_assets Hugging Face	Assets chính thức: Walker S2, task scenes, objects, foam/carton/conveyor.	https://huggingface.co/UBTECH-Robotics/challenge2026_assets
UBTECH challenge2026_dataset Hugging Face	Dataset chính thức hoặc mẫu để học cấu trúc dữ liệu.	https://huggingface.co/datasets/UBTECH-Robotics/challenge2026_dataset

8.2. Isaac Sim, synthetic data và perception model

Tài liệu	Dùng để học gì?	Link
NVIDIA Isaac Sim Developer Page	Tổng quan Isaac Sim, synthetic data, robot learning, simulation/testing.	https://developer.nvidia.com/isaac-sim
Isaac Sim Replicator Object / Synthetic Data	Tạo dữ liệu RGB, bounding boxes, segmentation masks bằng synthetic data generation.	https://docs.isaacsim.omniverse.nvidia.com/latest/action_and_event_data_generation/tutorial_replicator_object.html
NVIDIA Getting Started With Isaac Sim - Synthetic Dataset	Tutorial tạo synthetic dataset và domain randomization.	https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/synthetic-data-generation-for-perception-model-training-in-isaac-sim/04-generating-a-synthetic-dataset-using-a-replicator.html
NVIDIA Getting Started With Isaac Sim -	Tutorial fine-tune và validate model perception từ synthetic	https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-sim/latest/synthetic-data-generation-for-perception-model-training-in-isaac-sim/05-fine-tuning-and-validating-an-ai-perception-model.html

Tài liệu	Dùng để học gì?	Link
Fine-tuning Perception Model	data.	

8.3. Computer vision, segmentation, tracking và 6D pose

Tài liệu	Dùng để học gì?	Link
Ultralytics YOLO Documentation	Detection, segmentation, classification, pose, OBB và tracking tasks.	https://docs.ultralytics.com/
OpenCV Thresholding Tutorial	Segmentation cơ bản bằng threshold.	https://docs.opencv.org/master/db/d8e/tutorial_threshold.html
OpenCV Meanshift/Camshift Tracking	Object tracking cơ bản trong video.	https://docs.opencv.org/4.x/d7/d00/tutorial_meanshift.html
OpenCV solvePnP Documentation	Pose estimation từ 3D-2D correspondences.	https://docs.opencv.org/4.x/d5/d1f/calib3d_solvePnP.html
Open3D ICP Registration Tutorial	Point cloud alignment và pose refinement.	https://www.open3d.org/html/tutorial/Basic/icp_registration.html
Meta Segment Anything GitHub	Foundation segmentation model, hữu ích cho annotation/mask-assisted workflow.	https://github.com/facebookresearch/segment-anything
NVIDIA FoundationPose GitHub	6D pose estimation/tracking cho object mới dựa trên CAD/reference images.	https://github.com/NVlabs/FoundationPose

8.1. Lộ trình tự học trong 1 ngày

Thời lượng	Việc cần làm	Output
60 phút	Đọc lại Module 4: camera/RGB-D/transform; đọc phần 2.1-2.5 của Module 5.	Ghi 10 khái niệm chính.
90 phút	Làm Lab 2 classical mask/centroid/orientation trên frame mẫu.	overlay_centroid.png + CSV.
90 phút	Đọc Ultralytics hoặc OpenCV tracking tutorial; chạy inference/tracking thử.	detection/tracking overlay.
90 phút	Làm Lab 4 hoặc Lab 5 tùy guild.	pose/tracker report.
60 phút	Viết perception_interface.json và validator.	schema_validation_report.txt.
30 phút	Tự đánh giá theo rubric mini-project.	todo list cải thiện.

9. Câu hỏi tự kiểm tra

1. Vì sao bounding box thường chưa đủ cho manipulation?
2. Khi nào nên dùng threshold/mask thay vì YOLO?
3. Nếu depth có nhiều NaN trong mask, em xử lý thế nào?
4. Tại sao phải chia train/validation/test theo episode hoặc scene?
5. Task 2 cần thêm output nào mà Task 1 không bắt buộc?
6. Metric mAP cao nhưng end-to-end success thấp có thể do nguyên nhân gì?
7. failure_reason trong perception_output.json có tác dụng gì?
8. Làm sao phát hiện object pose không ổn định qua nhiều frame?

9. Khi dùng synthetic data từ Isaac Sim, cần randomize những yếu tố nào?
10. Perception output nào planner/motion module cần để quyết định grasp?